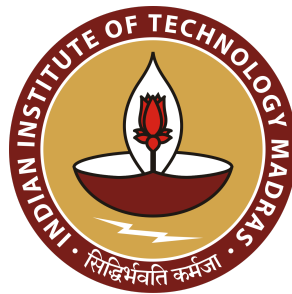


AXI4 IP Porting with Shakti SoC



DEVELOPED BY :
SHAKTI DEVELOPMENT TEAM @ IITM
SHAKTI.ORG.IN

Integration of IP supports AXI4

Any IP that supports AXI4 interface will be integrated at fpga_top level. It has an AXI4 slave interface which will be integrated with AXI4 Master interface of Shakti SoC which is the fast fabric. The slave must be added to fabric in Soc.bsv file and the memory mapping for the slave, adding of a slave number must be given in Soc.defines file.

In Soc.bsv file, add slave_num for the new slave

```
function Bit#(TLog#(`Num_Fast_Slaves)) fn_slave_map_fast (Bit#(`paddr) addr);
  Bit#(TLog#(`Num_Fast_Slaves)) slave_num = 0;
  if(addr >= `TCMBase && addr <= `TCMEnd)
    slave_num = `TCM_slave_num;
  ✨ else if(addr >= `AXI4Base && addr <= `AXI4End)
    ... slave_num = `AXI4_slave_num;
  else if(addr >= `ClintBase && addr <= `ClintEnd)
    slave_num = `Clint_slave_num;
`ifdef debug
  else if(addr >= `DebugBase && addr <= `DebugEnd)
    slave_num = `Debug_slave_num;
`endif
  else if(addr >= `BootBase && addr <= `BootEnd)
    slave_num = `Boot_slave_num;
  else if(addr >= `SlowBase && addr <= `SlowEnd)
    slave_num = `Slow_fabric_slave_num;
  else if(addr >= `PLICBase && addr <= `PLICEnd)
    slave_num = `Slow_fabric_slave_num;
`ifdef simulate
  else if(addr >= `SignBase && addr <= `SignEnd)
    slave_num = `Sign_slave_num;
`endif
  else
    slave_num = `FastErr_slave_num;
  return slave_num;
endfunction:fn_slave_map_fast
```

Declare axi4_master interface:

```
interface Ifc_chip_io;
    interface Ifc_sspi_io spi0_io;
    //    interface Ifc_sspi_io spi1_io;
    interface RS232 uart0_io;
    method I2C_out i2c0_out;           //I2c IO interface
    method I2C_out i2c1_out;           //I2c IO interface
    ✨ interface AXI4_Master_IFC#(`paddr, `ELEN, `0) axi4_master;
    interface IOCellSide iocell_io;
```

Connect axi4_master interface to fast fabric.

```
interface uart0_io = uart_cluster.uart0_io;
method i2c0_out = mixed_cluster.i2c0_out;           //I2c IO interface
method i2c1_out = mixed_cluster.i2c1_out;           //I2c IO interface
interface iocell_io = mixed_cluster.pinmuxtop_iocell_side; //GPIO IO interface
✨ interface axi4_master = fabric.v to slaves [`AXI4_slave_num];
method Action ext_interrupts(Bit#(2) i);
| wr_ext_interrupts <= i;
endmethod
endinterface;

endmodule: mkSoc
endpackage: Soc
```

In Soc.defines file,

```
`ifdef simulate
| `define Num_Fast_Slaves 8
`else
| `define Num_Fast_Slaves 7
`endif
`define TCM_slave_num 0
`define Clint_slave_num 1
`define Debug_slave_num 2
`define FastErr_slave_num 3
`define Slow_fabric_slave_num 4
`define Boot_slave_num 5
`define AXI4_slave_num 6
`ifdef simulate
| `define Sign_slave_num 7
`endif
```

1. Increment value of Num_Slaves by 1.
2. Add `define AXI4_slave_num *n* (replace *n* by the last slave number -1)
3. Change `define AXI4\_slave\_num *x* to `define Err_slave_num *x*+1 (*x* being the original value)

```

`define TCMBase          'h8000_0000
`define TCMEnd           'h8001_FFFF
`define MemoryBase      'h9000_0000
`define MemoryEnd       'h9001_FFFF
`define ClintBase       'h0200_0000
`define ClintEnd        'h020B_FFFF
`define DebugBase       'h0000_0000
`define DebugEnd        'h0000_0FFF
`define BootBase        'h0000_1000
`define BootEnd         'h0000_8FFF
`define AXI4Base         'h0001_0000
`define AXI4End          'h0001_00FF
`define SlowBase        'h0001_1300
`define SlowEnd         'h0004_7FFF
`ifdef simulate
    `define SignBase     'h0005_0000
    `define SignEnd      'h0005_000c
`endif

```

4. Add `define AXI4Base 'addr1 and `define AXI4End 'addr2 . Define addr1 and addr2 such that it does not conflict with other slave addresses.
- make generate_verilog : to generate updated verilog files
 - In fpga_top, the newly generated core's verilog file will now have the ports corresponding to axi4_master slave.

Generated mkDebugSoc.v with newly added ports for the axi4 slave:

```
i2c1_out_sda_out_en,  
axi4_master_AWVALID,  
axi4_master_AWADDR,  
axi4_master_AWPROT,  
axi4_master_AWLEN,  
axi4_master_AWSIZE,  
axi4_master_AWBURST,  
axi4_master_AWID,  
axi4_master_AWREADY,  
axi4_master_WVALID,  
axi4_master_WDATA,  
axi4_master_WSTRB,  
axi4_master_WLAST,  
axi4_master_WID,  
axi4_master_WREADY,  
axi4_master_BVALID,  
axi4_master_BRESP,  
axi4_master_BID,  
axi4_master_BREADY,  
axi4_master_ARVALID,  
axi4_master_ARADDR,  
axi4_master_ARPROT,  
axi4_master_ARLEN,  
axi4_master_ARSIZE,  
axi4_master_ARBURST,  
axi4_master_ARID,  
axi4_master_ARREADY,  
axi4_master_RVALID,  
axi4_master_RRESP,  
axi4_master_RDATA,  
axi4_master_RLAST,  
axi4_master_RID,  
axi4_master_RREADY,
```

- Open project (.xpr file) in Vivado GUI
- Connect the signals from the your IP to these newly generated ports ports using wires to integrate the IP to the Shakti SoC

Integrating in fpga_top.v:

Initialise the ports and wires that are required and part of your IP.

Within,
mkDebugSoc core(
);

Instantiate the ports for new slave and connect the wires from your IP AXI4 interface to this.

```
// AXI4 Master interface
→ → → .axi4_master_AWVALID(),
→ → → .axi4_master_AWADDR(),
→ → → .axi4_master_AWSIZE(),
→ → → .axi4_master_AWPROT(),
→ → → .axi4_master_AWLEN(),
→ → → .axi4_master_AWBURST(),
→ → → .axi4_master_AWID(),
→ → → .axi4_master_AWREADY(),
→ → → .axi4_master_WVALID(),
→ → → .axi4_master_WDATA(),
→ → → .axi4_master_WSTRB(),
→ → → .axi4_master_WLAST(),
→ → → .axi4_master_WID(),
→ → → .axi4_master_WREADY(),
→ → → .axi4_master_BVALID(),
→ → → .axi4_master_BRESP(),
→ → → .axi4_master_BID(),
→ → → .axi4_master_BREADY(),
→ → → .axi4_master_ARVALID(),
→ → → .axi4_master_ARADDR(),
→ → → .axi4_master_ARSIZE(),
→ → → .axi4_master_ARPROT(),
→ → → .axi4_master_ARLEN(),
→ → → .axi4_master_ARBURST(),
→ → → .axi4_master_ARID(),
→ → → .axi4_master_ARREADY(),
→ → → .axi4_master_RVALID(),
→ → → .axi4_master_RRESP(),
→ → → .axi4_master_RDATA(),
→ → → .axi4_master_RLAST(),
→ → → .axi4_master_RID(),
→ → → .axi4_master_RREADY(),
```

As the last step, add the initialized ports in the fpga_top with the appropriate mapping in constraints.xdc file.